1. Operating Systems and Processes

Approx. target: 2,500 characters of text on this page • 23 September 2026

An operating system is system software that manages computer hardware and provides a controlled environment for application programs. It acts as an intermediary between users and hardware resources such as the processor, main memory, storage devices, input devices, and network interfaces. The major goals of an operating system are convenience, efficient resource utilization, fairness, reliability, and protection. Instead of requiring every application to understand the details of a disk controller or memory-management unit, the operating system exposes abstractions such as files, processes, virtual memory, and sockets.

A process is a program in execution. A program stored on disk is passive, while a process is an active entity with a current execution state and allocated resources. A process normally contains program code, data, a stack, a heap, a program counter, CPU registers, and operating-system bookkeeping information. The process control block, or PCB, stores information required by the operating system to manage the process. Typical PCB information includes the process state, process identifier, program counter, CPU registers, scheduling information, memory-management information, accounting information, and I/O status.

Processes commonly move through several states. A new process is being created. A ready process is prepared to execute but is waiting for CPU time. A running process is currently executing instructions. A waiting or blocked process is waiting for an event such as completion of an I/O operation. A terminated process has finished execution. The operating system changes process states as scheduling decisions, interrupts, system calls, and external events occur. A context switch occurs when the CPU changes from one process to another. The operating system saves the state of the current process and restores the saved state of the next process.

Threads provide another level of execution within a process. Multiple threads belonging to the same process can share code, data, and many resources, while each thread maintains its own program counter, registers, and stack. Multithreading can improve responsiveness and allow parallel work on multicore processors. However, shared data introduces synchronization challenges. If two threads update the same variable without proper coordination, the final value can depend on the timing of their operations. This situation is called a race condition and can produce incorrect or inconsistent results.

Operating systems also provide system calls through which application programs request privileged services. A program may use system calls to create a process, open a file, allocate resources, communicate with another process, or interact with a device. System calls form a controlled boundary between user programs and the kernel. This boundary is important because unrestricted access to hardware could allow one program to corrupt another program's data or damage the operating system itself.

2. CPU Scheduling and Synchronization

Approx. target: 2,500 characters of text on this page • 23 September 2026

CPU scheduling determines which ready process receives the processor. In a system with multiple ready processes, the scheduler chooses one according to a scheduling policy. Important performance measures include CPU utilization, throughput, turnaround time, waiting time, and response time. CPU utilization describes how busy the processor is, while throughput measures the number of completed processes over a period. Turnaround time is the time from process submission to completion. Waiting time measures time spent waiting in the ready queue, and response time measures how quickly a system first responds to a request.

First-Come, First-Served scheduling executes processes in their order of arrival. It is simple and easy to implement, but a long CPU-bound process can make many shorter processes wait. This is known as the convoy effect. Shortest Job First selects the process with the smallest expected CPU burst. Under appropriate assumptions, non-preemptive SJF minimizes average waiting time. Shortest Remaining Time First is the preemptive form and can interrupt a running process when another process has a shorter remaining burst.

Priority scheduling chooses a process according to a priority value. A system may use either preemptive or non-preemptive priority scheduling. A major problem is starvation, in which a low-priority process waits for a very long time because higher-priority processes continue to arrive. Aging is a common solution: the priority of a waiting process is gradually increased. Round Robin scheduling assigns each ready process a time quantum. When the quantum expires, the running process is preempted and moved to the end of the ready queue if it still has work to perform. Round Robin is particularly useful for interactive workloads.

Synchronization is required when processes or threads share data. A critical section is a part of a program in which shared data or resources are accessed. A correct critical-section solution should provide mutual exclusion, progress, and bounded waiting. A mutex is a locking mechanism that permits one thread to enter a protected region at a time. Semaphores are integer-based synchronization objects accessed through atomic operations. A binary semaphore can provide mutual exclusion, while counting semaphores can represent multiple instances of a resource.

Synchronization primitives must be used carefully. Incorrect locking can lead to deadlock, in which processes wait forever for resources held by one another. Excessive locking can also reduce parallelism because threads spend too much time waiting. Good concurrent designs minimize shared mutable state, keep critical sections short, establish consistent lock ordering, and release resources reliably. These principles are important in operating systems, databases, web servers, and multithreaded applications.

3. Deadlocks and Memory Management

Approx. target: 2,500 characters of text on this page • 23 September 2026

A deadlock is a state in which a set of processes cannot continue because every process is waiting for a resource or event that can only be provided by another process in the same set. Four conditions are necessary for a classical resource deadlock: mutual exclusion, hold and wait, no preemption, and circular wait. Mutual exclusion means that a resource can be used by only one process at a time. Hold and wait means a process holds at least one resource while waiting for another. No preemption means resources cannot simply be taken away from a process. Circular wait means a cycle of processes exists where each process waits for a resource held by the next process.

Deadlock prevention attempts to ensure that at least one necessary condition cannot occur. Deadlock avoidance instead makes allocation decisions based on information about possible future resource requirements. The Banker's algorithm is a classic avoidance algorithm that checks whether granting a request keeps the system in a safe state. Deadlock detection allows deadlocks to occur and periodically searches for evidence of them. Recovery may involve terminating selected processes or preempting resources. Each strategy has costs, so operating systems choose mechanisms appropriate to the workload and resource model.

Memory management controls the use of main memory by processes. The operating system must allocate memory, track free and occupied regions, protect processes from one another, and translate addresses. Contiguous allocation assigns a contiguous physical region to each process. This approach can suffer from external fragmentation, where enough total free memory exists but it is divided into small holes. Paging reduces this problem by dividing logical memory into fixed-size pages and physical memory into fixed-size frames.

A page table maps a process's logical page numbers to physical frame numbers. When a CPU generates a logical address, the memory-management unit uses the page table to determine the corresponding physical address. Paging enables processes to have non-contiguous physical memory while maintaining a contiguous logical address space. A translation lookaside buffer, or TLB, can cache recent address translations and reduce the cost of repeated page-table lookups. Protection bits can also specify whether a page is readable, writable, executable, or accessible in a particular processor mode.

Virtual memory extends the abstraction of memory beyond physical RAM. A process can have a large virtual address space even when only some pages are resident in physical memory. With demand paging, pages are loaded when they are referenced. If a referenced page is absent, a page fault occurs. The operating system must locate the page on secondary storage, obtain a free frame or choose a victim page, load the required page, update the page table, and restart the interrupted instruction. Excessive page faults can cause thrashing and severely reduce performance.

4. Page Replacement, Files, and I/O

Approx. target: 2,500 characters of text on this page • 23 September 2026

Page replacement becomes necessary when a page fault occurs and all available physical frames are occupied. The operating system must choose a victim page to remove. FIFO replaces the page that has been in memory for the longest time. It is simple but can sometimes produce more page faults when additional frames are added, a phenomenon called Belady's anomaly. The Optimal algorithm replaces the page whose next use is farthest in the future. It provides the theoretical minimum page-fault count but cannot be implemented exactly for general workloads because future references are unknown.

Least Recently Used, or LRU, replaces the page that has not been referenced for the longest period. LRU uses the idea of temporal locality: pages used recently are more likely to be used again soon. Exact LRU can require substantial hardware or software support, so practical systems often use approximations. When evaluating page-replacement algorithms, a reference string and number of frames are specified. The algorithm is then simulated to count page faults and hits. This makes page replacement a common operating-system calculation problem.

A file system provides a structured way to store and retrieve data from persistent storage. Files have names and metadata such as size, ownership, permissions, timestamps, and allocation information. Directories organize files into a hierarchy. File operations typically include create, open, read, write, seek, close, rename, and delete. File allocation strategies determine how file blocks are placed on storage. Contiguous allocation is fast for sequential access but can suffer from external fragmentation. Linked allocation avoids external fragmentation but can make random access slower. Indexed allocation stores block references in an index structure.

Input/output management provides a uniform interface to different hardware devices. Device drivers translate operating-system requests into device-specific operations. Interrupts allow hardware to notify the processor that an event has occurred. Buffering temporarily stores data between components with different speeds or transfer sizes. Caching keeps frequently used information closer to the processor or application. Direct Memory Access, or DMA, allows devices to transfer blocks of data directly to memory with limited CPU intervention, improving efficiency for high-volume transfers.

Storage systems also require request scheduling. FCFS disk scheduling services requests in arrival order. SSTF selects the request closest to the current disk-head position and can reduce immediate seek distance, although it may starve distant requests. SCAN moves the disk head in one direction while serving requests and then reverses direction. C-SCAN services requests in one direction and returns to the beginning without servicing requests during the return movement. LOOK and C-LOOK are variants that avoid travelling to the physical end when no request exists there.

5. Security, Networking, and RAG Applications

Approx. target: 2,500 characters of text on this page • 23 September 2026

Operating-system protection controls access to resources, while security includes mechanisms for defending systems against unauthorized use and attacks. Authentication verifies an identity, while authorization determines which operations that identity may perform. Access-control lists can specify permissions for users or groups. The principle of least privilege means that a process or account should receive only the permissions necessary to complete its task. Process isolation, memory protection, secure boot mechanisms, file permissions, and auditing all contribute to a secure system.

Operating systems also provide networking facilities. Applications commonly use sockets to communicate over networks. TCP provides a reliable, ordered byte stream using acknowledgments, retransmission, sequencing, and congestion control. UDP is connectionless and has less protocol overhead but does not provide TCP's reliability guarantees. Network interfaces, routing tables, firewall rules, DNS configuration, and transport-layer ports are managed or exposed through operating-system facilities. Network services therefore depend on cooperation between applications, the operating-system network stack, and physical or virtual network hardware.

System performance should be evaluated using several metrics rather than a single number. CPU utilization indicates how much processor capacity is being used. Throughput measures completed work over time. Response time measures how quickly a system begins responding. Memory utilization, disk latency, I/O throughput, network throughput, cache hit rates, and page-fault frequency provide additional information. A system with high CPU utilization may still be slow if it spends significant time waiting for memory or I/O. Performance monitoring helps identify the actual bottleneck before optimization.

These operating-system concepts can also be used to test a Retrieval-Augmented Generation pipeline. A document loader extracts the text from a PDF or DOCX file. A recursive character text splitter divides the text into chunks. With a chunk size of 1000 characters and an overlap of 200 characters, consecutive chunks share some context. An embedding model such as Gemini Embedding converts each chunk into a vector. The vectors are stored in a vector database or similarity index. When a user asks a question, the question is embedded and compared with stored vectors to retrieve relevant chunks.

The retrieved chunks are then supplied to a language model together with the user's question and instructions. The language model generates an answer grounded in the retrieved context. This architecture separates retrieval from generation: the embedding model represents semantic information, the vector store performs similarity search, and the LLM produces the natural-language response. For IntelliLearn, this can support questions about uploaded study materials, summaries, quizzes, explanations, and flashcards. Retrieval quality depends on document extraction, chunk size, overlap, embedding quality, metadata, similarity search settings, and the final prompt. Testing with documents of different lengths helps measure indexing time, number of chunks, retrieval quality, and answer accuracy.